SIMATS ENGINEERING
ASSESSMENT TOOL

COURSE NAME: **Theory of Computation** **COURSE CODE:** **CSA13**

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5 **Total Marks:25**
Duration : 1 Hour

Application Based Question

Code of Conduct:

I B.Nynika(192311075) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Nynika

SIMATS ENGINEERING
ASSESSMENT TOOL

1.In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet Σ = {0,1} to validate passwords that **end with the pattern "01"**.Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

Problem Statement

Design a Deterministic Finite Automaton (DFA) over the alphabet

$$\Sigma = \{0, 1\}$$

that accepts all binary strings ending with **"01"**. Minimize the DFA and explain its real-world importance.

Step 1: DFA Definition

The DFA is defined as

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where

- **Q = {q_0, q_1, q_2}**
- **Σ = {0,1}**
- **Start State = q_0**
- **Final State = {q_2}**

Step 2: Transition Table

Present State	Input 0	Input 1
→ q_0	q_1	q_0
q_1	q_1	q_2
*q_2	q_1	q_0

Step 3: Working

- q_0 → Initial state
- q_1 → Last symbol is 0
- q_2 → Last two symbols are 01 (Accept)

Example

Input	Result
01	Accepted
1001	Accepted

Input	Result
101	Accepted
111	Rejected
1100	Rejected

Step 4: State Diagram

(Generate DFA diagram image.)

Step 5: DFA Minimization

Check equivalent states.

- q_0 → Non-final
- q_1 → Non-final
- q_2 → Final

Each state behaves differently.

Therefore,

No states are equivalent.

Hence,

The DFA is already minimal (3 states).

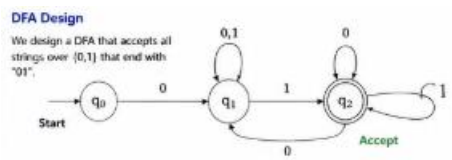
Real-World Analysis

Minimized DFA provides:

- Less memory usage
- Faster password verification
- Lower hardware cost
- Reduced power consumption
- High-speed authentication

Applications

- ATM machines
- Embedded systems
- Smart cards
- Security authentication modules



SIMATS ENGINEERING
ASSESSMENT TOOL

2.In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet Σ = {a, b}.Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

Problem Statement

Design a regular expression that accepts only

aaaabb

over

$$\Sigma = \{a, b\}$$

Regular Expression

aaaabb

or

aaaa bb

Both represent exactly one string:

aaaabb

Equivalent Finite Automaton

States

q0→q1→q2→q3→q4→q5→q6

Inputs

a a a a b b

Final state

q6

Any incorrect symbol goes to a dead state.

Transition Table

State	a	b
→q0	q1	Dead
q1	q2	Dead
q2	q3	Dead
q3	q4	Dead
q4	Dead	q5
q5	Dead	q6
*q6	Dead	Dead

Real-World Analysis

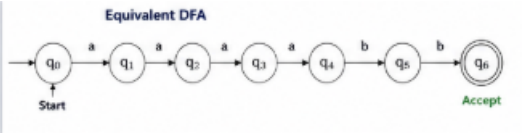
Regular expressions are used in

- **Login validation**
- **Email validation**
- **Phone number validation**
- **Password checking**
- **Form validation**

- **SQL injection prevention**
- **XSS filtering**

Advantages

- **Fast execution**
- **Easy implementation**
- **High performance**
- **Low memory usage**



3.In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression `a*b*` can be converted into an equivalent finite automaton (FA).Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

Problem Statement

Convert

a*b*

into an equivalent finite automaton.

Meaning

a*

means

Zero or more a's

b*

means

Zero or more b's

Examples accepted

ε

a

aa

aaa

b

bb

ab

aab

aaabbb

Rejected

aba

bab

abba

Equivalent FA

States

- q_0
- q_1

Transitions

- q_0 loops on a
- q_0 goes to q_1 on b
- q_1 loops on b

Accepting states

q_0

q_1

Transition Table

State	a	b
→*q_0	q_0	q_1
*q_1	Dead	q_1

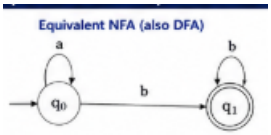
Why RE and FA are Equivalent

Every Regular Expression can be converted into an NFA.
Every NFA can be converted into a DFA.
Every DFA can be converted back into a Regular Expression.
Hence,
Regular Expressions and Finite Automata recognize exactly the **same class of languages (Regular Languages).**

Real-Time Applications

Used in

- Intrusion Detection Systems
- Network monitoring
- Log analysis
- Search engines
- Lexical analyzers
- Text editors
- Compilers



4.In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language L= { $a^n b^n$ | n ≥ 0 } cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

Problem Statement

Show that

$$L = \{a^n b^n \mid n \geq 0\}$$

is not regular.

Pumping Lemma

Assume

L is regular.

Then there exists a pumping length

p

Choose

w=a^p b^p

Clearly

|w|≥p

Split

w=xyz

where

- |xy|≤p
- |y|>0

Since

|xy|≤p

the substring **y** contains only a's.

Let

y=a^k

Pump down

i=0

New string

xz=a^(p-k)b^p

Now,

Number of a's ≠ Number of b's

Therefore,

xz ∉ L

Contradiction.

Hence,

$$L = \{a^n b^n\}$$

is **not regular**.

Implications in Compiler Design

Finite automata cannot recognize

- Balanced parentheses
- Equal numbers of symbols
- Nested loops
- Recursive structures

Therefore,

Compilers use

- Context-Free Grammars (CFGs)
- Pushdown Automata (PDAs)

for syntax analysis.

5.Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata

5. Closure Properties of Regular Languages

Problem Statement

Explain how closure properties help combine multiple detection rules.

Closure Properties

If

L_1

and

L_2

are regular languages,

then the following are also regular.

1. Union

$$L_1 \cup L_2$$

Accept strings in either language.

---

2. Intersection

$$L_1 \cap L_2$$

Accept strings common to both languages.

---

3. Complement

$$\bar{L}$$

Accept strings not in the language.

---

4. Concatenation

$$L_1 L_2$$

Strings from L_1 followed by strings from L_2.

---

5. Kleene Star

$$L^*$$

Zero or more repetitions.

Why the Resulting Language Remains Regular

Regular languages are closed under:

- Union
- Intersection
- Complement
- Concatenation
- Kleene Star

Therefore,

Combining regular languages always produces another **regular language**, which can still be recognized efficiently by finite automata.

Real-World Applications

Intrusion Detection

Combine attack signatures using union and intersection.

Spam Filtering

Combine multiple spam rules.

Input Validation

Validate multiple input formats simultaneously.

Firewall Systems

Merge several filtering policies.

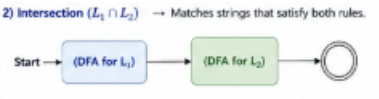
SIMATS ENGINEERING
ASSESSMENT TOOL

Lexical Analysis

Combine token definitions into a single scanner.

Conclusion

Closure properties allow multiple pattern-matching rules to be combined while preserving regularity. This ensures efficient implementation using DFAs or NFAs, making them suitable for high-speed applications such as intrusion detection, spam filtering, input validation, compilers, and network security.



SIMATS ENGINEERING
ASSESSMENT TOOL

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

2. Can enforce rules like:

- **Email format**
- **Password rules**
- **Input structure**